

CloudShield AI

27 — AWS & Terraform Specification

Version 1.0 | Product-Level Infrastructure Engineering Specification

CloudShield AI / CloudSpartanX

Document purpose: Define the AWS deployment architecture, Terraform standards, environment topology, networking, compute, data services, queues, object storage, IAM, secrets, observability, backup, disaster recovery, CI/CD integration, and infrastructure security baseline for CloudShield AI.

Canonical principle: AI proposes. Deterministic systems validate. Governance decides. Cloud APIs execute. Evidence records. Trusted state is earned.

1. Purpose and Scope

This specification translates the CloudShield AI product and subsystem architecture into a concrete AWS infrastructure and Infrastructure-as-Code (IaC) design. AWS is the primary cloud provider for the MVP.

- Define the AWS resource topology required by CloudShield AI.
- Define Terraform repository/module/state standards.
- Define development, test, staging, and production environment separation.
- Define network, compute, database, object storage, queue, secrets, IAM, and observability architecture.
- Define how DAIL workers, AI services, discovery workers, and application services are deployed safely.
- Define cloud-account and tenant boundaries.
- Define infrastructure security, backup, recovery, scaling, and cost controls.
- Provide implementation-ready infrastructure conventions without prematurely locking the product into unnecessary AWS services.

MVP should favor a small number of managed AWS services and a modular application deployment over premature infrastructure complexity.

2. Infrastructure Architecture Principles

- Infrastructure is reproducible through Terraform.
- Production infrastructure changes are reviewed and planned before application.
- AWS IAM follows least privilege and explicit separation of discovery, application, worker, and promotion authorities.
- Sensitive services remain private wherever practical.
- Stateful services use managed AWS capabilities unless a documented reason requires otherwise.
- Environment boundaries are explicit and prevent accidental cross-environment access.
- CloudShield AI application logic must not depend on Terraform implementation details.
- Terraform resources are tagged consistently for ownership, environment, tenant boundary where appropriate, cost attribution, and lifecycle.
- Infrastructure drift is detectable and recoverable.

- High-risk infrastructure changes use controlled CI/CD and, where they affect customer environments, DAIL governance.

3. AWS Deployment Topology

Layer	AWS Capability	MVP Role
Edge	DNS/CDN/WAF as required	Public application entry point and basic protection.
Application	Containerized API/application service	Core CloudShield AI product APIs and UI backend.
Workers	Containerized/background workers	Discovery, security analysis, AI, DAIL/verification, reporting.
Queue	Managed queue	Asynchronous jobs and decoupling.
Database	Managed relational database	Canonical transactional system of record.
Object Storage	Managed object storage	Evidence, reports, large artifacts, backups where applicable.
Secrets	Managed secrets service	Provider/cloud/application credentials.
Observability	Logs, metrics, alarms	Operational and security telemetry.
IAM	Roles/policies	Workload and operator authorization.
Networking	VPC/private subnets/security groups	Isolation and controlled service communication.

4. Canonical MVP Topology

Recommended logical deployment:

Internet →DNS/CDN/WAF →Public Load Balancer/API Entry →Private Application Service →Private Workers →Queue →Managed Database/Object Storage/Secrets. Application and workers access AWS APIs using dedicated workload identities. Administrative access is separate from runtime identities.

Component	Placement	Exposure
Web/API entry	Edge/public subnet or managed edge	Public via controlled endpoint.
Application service	Private subnet	No direct public network exposure.
Discovery worker	Private subnet	No inbound public access.
Security/Risk worker	Private subnet	No inbound public access.
AI worker	Private subnet	Outbound only to approved provider endpoints.
DAIL/Verification worker	Private subnet	No inbound public access; tightly scoped cloud permissions.
Reporting/Evidence worker	Private subnet	No inbound public access.
Database	Private subnet/managed	No public access.
Object storage	AWS service endpoint/private access where practical	No public bucket access.
Queue	Managed AWS service	Application/service identities only.

5. AWS Account and Environment Strategy

5.1 Environments

Environment	Purpose	Data Rule
Development	Local/integration development	Synthetic or non-production data.
Test	Automated integration/security testing	Controlled test data.
Staging	Production-like validation	No unrestricted production secrets/data.
Production	Customer-facing operation	Production data and strict change control.

5.2 Account strategy

For an initial MVP, separate AWS accounts for production and non-production are strongly preferred. If organizational constraints require fewer accounts, environments must still use distinct VPCs, IAM roles, state backends, secrets, and deployment pipelines.

- Never reuse production credentials in development or test.
- Production Terraform state must be isolated from non-production state.
- Production deployment roles must not be usable by ordinary application workloads.
- Cross-account access, where used, must be explicit and least-privileged.

6. VPC and Network Architecture

6.1 Network zones

Zone	Contents	Rule
Public/edge	Load balancer or controlled ingress	Only required inbound ports.
Application private	API/application services	No direct internet ingress.
Worker private	Background/DAILY/AI workers	No direct inbound internet.
Data private	Database and private data services	No public access.
Management	Optional controlled administrative paths	Restricted operator access.

6.2 Network controls

- Use security groups as stateful service-level controls.
- Keep database access restricted to explicitly authorized application/worker security groups.
- Avoid broad 0.0.0.0/0 ingress except where required at an edge component.
- Control outbound access from AI workers and other workloads that require external APIs.
- Use VPC endpoints/private connectivity for AWS services where cost and architecture justify it.
- Do not treat network placement as a substitute for IAM authorization.

7. Compute Architecture

MVP should use a managed container runtime or equivalent managed compute model that supports private workloads, autoscaling, IAM roles, health checks, and controlled deployment. The exact AWS compute service can be selected during implementation based on team constraints; application architecture remains container-oriented.

Workload	Characteristics	Scaling
API/Application	Stateless HTTP/API workload	Horizontal scaling.
Discovery	Long-running cloud inventory jobs	Worker concurrency.
Security/Risk	CPU/query-oriented analysis	Queue-driven scaling.
AI	External provider calls; variable latency	Concurrency + provider quotas.
DAIL/Verification	Safety-critical jobs	Controlled concurrency; explicit isolation.
Reporting	Artifact generation	Queue-driven batch scaling.

- Container images must be immutable by digest/version in production.
- Health checks must distinguish process health from dependency readiness.
- Workers must support graceful shutdown and job acknowledgement semantics.
- High-risk DAIL workers should have separate IAM roles from generic application workers.

8. Database Architecture

The MVP system of record should use a managed relational database because CloudShield AI has strongly related transactional entities, lifecycle state, authorization boundaries, candidate versions, evidence metadata, and audit records.

8.1 Database requirements

- Private network access only.
- Encryption at rest enabled.
- Automated backups enabled.
- Deletion protection appropriate for production.
- Schema migrations controlled through application CI/CD.
- Connection pooling and bounded connection limits.
- Tenant-aware indexes and constraints where appropriate.
- Audit-sensitive records protected by application authorization and database constraints.

8.2 Data placement

Data	Primary Store
Core domain entities	Relational database
Lifecycle/state	Relational database
Dependency edges	Relational database initially
AI request metadata	Relational database
Large AI artifacts	Object storage
Evidence artifacts	Object storage + relational metadata
Audit events	Relational database and/or append-oriented log sink
Jobs	Queue + durable job metadata in relational database

9. Object Storage Architecture

- Use private object storage buckets for evidence, reports, large configuration artifacts, and generated outputs.
- Block public access.
- Encrypt objects at rest.
- Use lifecycle policies for retention classes.

- Use object versioning for evidence/artifacts where appropriate.
- Use separate prefixes/buckets by environment and data class.
- Avoid putting secrets into object storage.
- Use presigned URLs only for explicitly authorized, time-limited user access.

Artifact Class	Retention Approach
Security evidence	Longer retention according to product policy.
Reports	Configurable tenant/product retention.
Transient job artifacts	Short lifecycle.
Backups	Managed backup retention policy.
Evaluation artifacts	Versioned, access-controlled, potentially separate from production data.

10. Queue and Asynchronous Processing

Long-running discovery, analysis, AI, verification, and reporting work should be represented as asynchronous jobs.

Job Type	Queue Characteristics
Discovery	Potentially high volume; idempotent.
Security analysis	Parallelizable; bounded concurrency.
AI	Provider rate-aware; retry/backoff.
Verification	Safety-critical; controlled concurrency and explicit state.
Promotion/reconciliation	Low volume; strongest authorization and audit.
Reporting	Batch; artifact-oriented.

- Queue messages contain references/IDs rather than unnecessary sensitive payloads.
- Workers must validate tenant/scope and job ownership before processing.
- Use visibility timeouts/acknowledgement semantics appropriate to job duration.
- Poison messages must be quarantined/dead-lettered.
- Job processing must be idempotent.
- A queue retry must never bypass lifecycle authorization or DAIL state checks.

11. IAM Architecture

11.1 Runtime roles

Role	Primary Permissions	Authority
CloudShieldApplicationRole	Read/write product data, queue, object metadata	No customer cloud mutation.
DiscoveryRole	Read-only cloud inventory/configuration permissions	Discovery only.
SecurityAnalysisRole	Product data + security evaluation inputs	No cloud mutation.
AIWorkerRole	AI service, queue, controlled data access	No unrestricted cloud mutation.
DAILVerificationRole	Required read/verification APIs	Only approved verification scope.
PromotionRole	Explicitly required cloud mutation operations	Highest runtime authority; heavily restricted.

Role	Primary Permissions	Authority
ReportingRole	Read authorized data + write reports	No infrastructure mutation.

11.2 IAM rules

- Separate discovery and promotion identities.
- Do not give application containers broad administrator permissions.
- Use workload identities/roles rather than embedded AWS access keys.
- Restrict permissions by action and resource where AWS supports resource-level constraints.
- Use explicit deny controls for protected operations where useful.
- Audit privileged role assumption and promotion operations.
- Production operator roles are separate from runtime roles.

12. Customer Cloud Access Architecture

CloudShield AI must distinguish its own AWS infrastructure identity from customer cloud identities. Customer environments are accessed through explicit CloudConnection configurations and scoped cloud roles/assume-role relationships.

Boundary	Design
CloudShield AWS account	Hosts the product.
Customer AWS account	Contains customer infrastructure being discovered/analyzed.
Connection	Stores metadata and secure reference to access configuration.
Discovery role	Customer-side least-privilege read role.
Execution role	Separate customer-side role for governed changes where enabled.
Trust policy	Explicitly restricts which CloudShield identity may assume the customer role.
Scope	Account/region/resource permissions constrained by policy.

MVP should default to read-only customer access for discovery and analysis. Governed mutation can be enabled only after the complete DAIL verification/promotion path is implemented.

13. Secrets and Key Management

- Use a managed secrets service for application/provider secrets.
- Use IAM roles rather than static AWS keys for AWS workload access.
- Customer connection secrets must never be returned to the browser.
- Encrypt secrets with managed key-management capabilities.
- Rotate credentials according to policy and provider requirements.
- Do not store secrets in Terraform variables, state, logs, container images, or source control unless the secret-management architecture explicitly protects them.
- Terraform must avoid exposing sensitive values in outputs.

14. Terraform Architecture

14.1 Repository structure

Recommended repository structure:

Path	Purpose
infra/	Terraform root.
infra/modules/network/	Reusable VPC/network module.
infra/modules/compute/	Application/worker compute modules.
infra/modules/database/	Managed database module.
infra/modules/storage/	Object storage module.
infra/modules/queue/	Queue/dead-letter module.
infra/modules/iam/	Workload and deployment roles.
infra/modules/observability/	Logs, alarms, dashboards.
infra/modules/secrets/	Secret metadata/access policies.
infra/env/dev/	Development root configuration.
infra/env/test/	Test root configuration.
infra/env/staging/	Staging root configuration.
infra/env/prod/	Production root configuration.
infra/policies/	Infrastructure policy/validation rules.

14.2 Module rules

- Modules should expose stable interfaces and avoid environment-specific assumptions.
- Do not hide security-sensitive IAM policy inside opaque generic modules.
- Keep modules focused on one infrastructure responsibility.
- Pin provider versions and module versions.
- Use variables for intentional environment differences.
- Prefer explicit resource naming and tagging.
- Document inputs, outputs, security implications, and dependencies.

15. Terraform State Management

- Remote state is mandatory for shared environments.
- Production state must be isolated from all other environments.
- State storage must use encryption and restricted access.
- State locking/concurrency control must be enabled.
- Only CI/CD deployment roles and tightly controlled operators can modify production state.
- State files must be treated as sensitive because they can contain resource metadata and, depending on configuration, sensitive values.
- State backup/recovery procedures must be tested.

Environment	State Isolation
Development	Separate state.
Test	Separate state.
Staging	Separate state.
Production	Dedicated protected state and access policy.

16. Terraform Workflow

1. Developer modifies Terraform/module configuration.
2. Formatting and static validation run.
3. Provider/module version checks run.
4. Security/IaC scanning runs.
5. Terraform plan is generated.
6. Plan is reviewed for unexpected changes.
7. Approved environments apply through controlled CI/CD.
8. Post-apply validation confirms critical resources and security settings.
9. Drift detection runs periodically.
10. Unexpected production drift is investigated before corrective apply.

17. Infrastructure Security Controls

Control	MVP Requirement
Public storage	Disabled.
Database public access	Disabled.
Encryption	Enabled for sensitive data stores.
IAM	Least privilege.
Logging	Centralized application/security logs.
Network	Private workloads.
Secrets	Managed secret storage.
Image security	Vulnerability scanning.
Terraform	Plan/review before apply.
Drift	Detection and alerting.
Backups	Automated production backups.
Deletion protection	Enabled for critical production stateful resources where appropriate.

18. Observability Architecture

- Centralize application and worker logs.
- Include environment, service, job ID, tenant/scope reference where safe, correlation ID, and severity.
- Never log raw secrets or cloud credentials.
- Emit metrics for queue depth, job failures, API latency, AI usage, verification failures, and promotion events.
- Create alarms for infrastructure health, database availability, queue backlog, privileged actions, and security anomalies.
- Retain security-relevant audit/evidence separately from noisy debug logs where appropriate.

Signal	Example
Infrastructure	CPU/memory, container restarts, database health.
Application	5xx rate, latency, request rate.
Workers	Job success/failure, processing duration.
Queue	Depth, age of oldest message, DLQ count.
AI	Provider errors, latency, token/cost usage.
DAIL	Verification failures, blocked promotions, reconciliation events.
Security	Auth failures, IAM anomalies, unauthorized access attempts.

19. DNS, Edge, and WAF

- Use managed DNS for application endpoints.
- Terminate TLS at a controlled edge/load-balancing layer.
- Use a web application firewall where internet exposure warrants it.
- Apply request-size, rate, and abuse controls at the edge and application layers.
- Do not expose internal service endpoints publicly.
- Separate customer-facing endpoints from administrative endpoints.

20. Container and Artifact Security

- Build images in controlled CI/CD.
- Scan images and dependencies for vulnerabilities.
- Do not embed secrets in images.
- Run containers with least privilege and non-root execution where supported.
- Pin base images and dependencies.
- Use immutable image tags/digests in production.
- Restrict production deployment to trusted build artifacts.
- Retain provenance metadata for deployed artifacts.

21. CI/CD Integration

Stage	Terraform/Application Control
Validate	Formatting, syntax, static analysis.
Security scan	IaC, dependency, container, secret scanning.
Plan	Generate environment-specific plan.
Review	Human approval for protected environments.
Apply	Controlled deployment identity.
Verify	Health/security checks.
Monitor	Post-deployment telemetry.
Rollback	Revert application/configuration or execute documented infrastructure recovery.

CI/CD identities must be separate from runtime identities. A compromised application container must not automatically gain Terraform deployment authority.

22. Deployment Strategy

- Development deploys may be automated.
- Test deploys should run automatically after successful validation.
- Staging deploys should require successful integration/security tests.
- Production infrastructure changes require protected branch/review and controlled apply.
- High-risk changes should use explicit approval and maintenance/change windows where applicable.
- Application and infrastructure releases should be independently traceable.
- Infrastructure changes affecting CloudShield AI's own production security boundary receive stronger review than ordinary application changes.

23. DAIL Infrastructure Integration

DAIL workers require stronger infrastructure isolation because they participate in verification and promotion workflows.

DAIL Component	AWS Requirement
DAIL worker	Private workload; dedicated IAM role.
Verification engine	Explicitly scoped AWS API permissions.
Promotion service	Separate high-privilege role; no generic application access.
Evidence writer	Write access only to designated evidence stores.
Queue	Dedicated or logically isolated safety-critical queue.
Audit	Mandatory structured security/lifecycle events.
Reconciliation worker	Read/diagnostic permissions plus tightly controlled recovery authority.

The infrastructure must make it technically difficult for an ordinary API/application compromise to jump directly into promotion authority.

24. AI Infrastructure Integration

- AI workers run without customer-cloud mutation permissions.
- AI provider credentials are separate from AWS customer access credentials.
- Outbound network access is restricted to approved provider endpoints where practical.
- AI workloads can be disabled independently using configuration/feature controls.
- AI worker autoscaling must respect provider quotas and product budgets.
- Model/provider secrets remain in managed secret storage.

25. Backup and Disaster Recovery

Asset	Baseline
Relational database	Automated backups + tested restore.
Object storage	Versioning/lifecycle and appropriate replication/backup.
Terraform state	Protected remote state + recovery procedure.
Configuration	Version-controlled source/configuration.
Secrets	Managed secret backup/recovery according to service capability.
Container artifacts	Retained immutable production versions.

- Recovery procedures must restore both infrastructure and application dependencies.
- Database restore must be tested in a non-production environment.
- Critical evidence must remain recoverable without weakening its integrity guarantees.
- Recovery must not automatically promote or replay infrastructure changes.
- RTO/RPO values should be established per deployment tier before production scale-up.

26. Scaling and Capacity

- Scale stateless API workloads horizontally.

- Scale workers based on queue depth and processing latency.
- Bound AI concurrency to provider and budget constraints.
- Scale database capacity based on measured workload rather than speculative sizing.
- Monitor object-storage and queue growth.
- Apply per-tenant quotas to prevent one tenant from exhausting shared infrastructure.
- Load-test discovery and analysis workloads before production expansion.

27. Cost Management

Area	Control
Compute	Autoscaling and right-sized workloads.
Database	Measured capacity and scheduled scaling where appropriate.
AI	Per-tenant/task budgets and usage telemetry.
Storage	Lifecycle policies and retention controls.
Logs	Retention tiers and volume controls.
Network	Private connectivity evaluated against cost/benefit.
Environments	Automatic shutdown/scaling for non-production where appropriate.

Cost optimization must not remove security-critical controls such as encryption, backups, isolation, or auditability.

28. Infrastructure Policy and Tagging

Tag	Purpose
Project	CloudShield AI identification.
Environment	dev/test/staging/prod.
Service	Application/worker/component owner.
ManagedBy	Terraform ownership.
CostCenter	Cost attribution.
DataClass	Sensitive/internal/public classification where appropriate.
Lifecycle	Ephemeral/managed/critical.
Owner	Responsible team/service.

Tenant IDs should not be placed in public resource names or tags when doing so would expose customer information. Tenant attribution belongs in protected application metadata unless a private infrastructure convention explicitly requires otherwise.

29. Drift Detection and Reconciliation

- Run scheduled Terraform plan/drift detection for managed infrastructure.
- Alert on unexpected security-group, IAM, storage-public-access, encryption, or database-network changes.
- Do not blindly overwrite manually changed production resources.
- Classify drift as expected, unauthorized, or ambiguous.
- Preserve evidence of significant drift.
- Where drift affects CloudShield AI safety assumptions, trigger the appropriate reconciliation and verification workflow.

30. Infrastructure Threats

Threat	Mitigation
Terraform credential compromise	Protected CI role, short-lived credentials, least privilege.
State exposure	Encrypted restricted remote state.
Public database/storage	Private networking and public-access blocks.
Over-privileged workload	Dedicated least-privilege IAM roles.
Container compromise	Non-root, scanning, immutable artifacts, restricted IAM.
CI/CD compromise	Separate deployment/runtime identities, protected branches.
Drift	Detection, alerting, controlled reconciliation.
Cross-environment access	Account/state/role isolation.
Promotion privilege escalation	Separate DAIL promotion identity and policy gates.
Secret leakage	Managed secrets + scanning + no secret logs.

31. Terraform Security Acceptance Criteria

11. Production Terraform state is isolated and access-controlled.
12. No production workload has Terraform deployment permissions.
13. Database and object storage are not publicly accessible.
14. Sensitive stores are encrypted.
15. Runtime IAM roles follow least privilege and are separated by authority.
16. Terraform plans are reviewed before protected production applies.
17. IaC and dependency security scans run in CI.
18. Critical infrastructure drift is detected and alerted.
19. Secrets are not committed to source control or embedded in images.
20. DAIL promotion authority is technically separated from ordinary application authority.

32. Functional Infrastructure Acceptance Criteria

21. A fresh environment can be provisioned from version-controlled Terraform.
22. Application and workers can communicate through private infrastructure paths.
23. Asynchronous jobs can survive worker restarts through durable queue/job state.
24. Evidence artifacts can be stored and retrieved under tenant-aware authorization.
25. Production database backups can be restored successfully in a controlled test.
26. AI workers can reach approved model providers without receiving customer-cloud mutation authority.
27. DAIL verification and promotion workloads run under dedicated identities.
28. Infrastructure changes can be planned, reviewed, applied, and audited through CI/CD.
29. Environment-specific configuration does not require copying production secrets into source code.

33. Implementation Sequence

30. Create AWS account/environment boundaries.
31. Create Terraform remote state and CI/CD deployment roles.
32. Provision networking and security groups.
33. Provision relational database and encryption.
34. Provision private object storage and lifecycle policies.
35. Provision queues and dead-letter queues.

36. Provision application and worker compute.
37. Provision IAM workload roles.
38. Provision secrets and provider configuration.
39. Provision observability, alarms, and audit logging.
40. Deploy CloudShield AI application services.
41. Deploy discovery/security/AI/DAIL workers.
42. Validate end-to-end connectivity and authorization.
43. Run security, backup, restore, load, and drift tests.

34. MVP Infrastructure Baseline

- AWS as primary cloud provider.
- Separate production and non-production environments/accounts where practical.
- Terraform-managed infrastructure.
- Private VPC architecture.
- Managed relational database.
- Private object storage.
- Managed queue with dead-letter handling.
- Containerized application and workers.
- Dedicated IAM roles for application, discovery, AI, verification, and promotion.
- Managed secrets.
- Encryption at rest and TLS in transit.
- Centralized logs/metrics/alarms.
- Automated database backups.
- Protected Terraform state.
- CI/CD plan/review/apply workflow.
- No direct AI-to-cloud mutation authority.
- DAIL promotion authority technically isolated.

35. Future Infrastructure Evolution

- Multi-account organizational landing zone.
- Multi-region disaster recovery.
- Private service-to-service networking and stronger segmentation.
- Dedicated graph infrastructure at scale.
- Managed Kubernetes only if workload scale/operational needs justify it.
- Private model-serving infrastructure for local/open-weight LLMs.
- Advanced SIEM/security analytics.
- Cross-region evidence replication.
- Automated policy-as-code infrastructure compliance.
- Confidential computing or stronger workload isolation where customer requirements justify it.

36. Traceability

Source	Infrastructure Dependency
00 — Master System Specification	Product boundary, AWS MVP choice, trust model, DAIL positioning.
01 — Software Requirements Specification	Cloud onboarding, security, reliability, performance,

Source	Infrastructure Dependency
	observability, integrations.
02 — System Architecture Specification	Deployment topology, services, data stores, workers, queues, cloud integration.
03 — Functional Specification	Discovery, analysis, AI, remediation, verification, promotion, jobs.
04 — Product & UX Specification	Operational status, job state, errors, trusted state presentation.
05 — Domain & Data Model	Assets, observations, candidates, evidence, jobs, audit, policies.
06 — State & Lifecycle Specification	Durable state, concurrency, verification, promotion, reconciliation.
07 — Identity & Dependency Specification	Stable resource identity and dependency data persistence.
08 — Security & Threat Model	IAM, secrets, network, encryption, tenant isolation, supply chain.
09 — Intelligence & AI Architecture	AI workers, provider boundary, evaluation and routing.
10 — LLM Integration	Provider credentials, AI worker architecture, quotas, retries.
11 — AI Safety & Guardrails	AI kill switch, tool restrictions, no direct cloud mutation.
DAIL specifications	Verification/promotion roles, evidence, lifecycle, Terraform/AWS safety integration.

37. Engineering Decisions

Decision	Rationale
AWS primary MVP platform	Provides managed primitives for secure cloud-native deployment.
Terraform as IaC source of truth	Reproducibility, reviewability, and drift detection.
Managed relational DB	Matches transactional domain/lifecycle model.
Private application/data workloads	Reduces public attack surface.
Dedicated runtime IAM roles	Prevents authority concentration.
Separate promotion identity	Protects highest-impact operation.
Container-oriented deployment	Supports modular application + worker architecture.
Queue-driven background work	Improves resilience and isolates long-running jobs.
No premature Kubernetes requirement	Avoids operational complexity before scale requires it.
Production/non-production isolation	Reduces accidental data and privilege crossover.

38. Definition of Done

- Terraform repository structure and environment strategy are implemented.
- Remote state and locking/concurrency protection are configured.
- Network, compute, database, storage, queue, secrets, IAM, and observability modules exist.
- Production and non-production boundaries are enforced.
- Runtime identities are separated by authority.
- DAIL verification/promotion identities are isolated.
- Application workloads are private.
- Sensitive stores are encrypted and non-public.
- CI/CD performs validation, security scans, plan, controlled apply, and post-deployment checks.

- Backups and restore testing are operational.
- Infrastructure drift detection is configured.
- Cost/usage telemetry exists.
- Infrastructure security acceptance tests pass.
- A clean environment can be provisioned reproducibly from version-controlled Terraform.

39. Glossary

Term	Definition
AWS	Primary cloud infrastructure provider for the CloudShield AI MVP.
Terraform	Infrastructure-as-Code system used as the reproducible infrastructure source of truth.
VPC	AWS virtual network boundary containing CloudShield AI workloads.
Runtime Role	IAM role assumed by an application or worker workload.
Promotion Role	Highly restricted identity used only for governed infrastructure promotion.
Remote State	Protected Terraform state stored outside local developer machines.
Drift	Difference between declared Terraform infrastructure and actual deployed infrastructure.
DLQ	Dead-letter queue for jobs that cannot be processed successfully.
DAIL	Deterministic AI Infrastructure Lifecycle subsystem.
Customer Cloud	AWS account/environment being discovered, analyzed, or governed by CloudShield AI.
Trusted State	Infrastructure state earned after successful governed promotion.